Company: Microsoft
Interview Type: CASE_STUDY

Q1: Designing a Global Distributed Rate Limiter for Azure Front Door

Difficulty: Hard

Tags: Distributed Systems, Scalability, Redis, Concurrency

Problem Statement: Design a system to limit API requests across 100+ global edge locations. The system must support per-customer limits (e.g., 5000 requests/min), handle traffic bursts, and maintain low latency (< 2ms overhead) while ensuring high availability and eventual consistency across regions.

Expected Approach: Implement a centralized Redis cluster to track global counters for each user/API key. Every edge node performs a network call to the central Redis to increment and check the limit before serving the request.

Optimized Approach: Use a multi-tier "Leaking Bucket" algorithm. Implement local in-memory counters at each edge node for immediate enforcement. Use a background process to synchronize local tallies with a regional aggregator using a "Sync-on-Delta" strategy. Regional aggregators then sync with a global store (Cosmos DB) asynchronously. Use consistent hashing to distribute user keys across aggregators to prevent hot spots.

Time Complexity: O(1) for local checks; O(1) for asynchronous global sync.

Space Complexity: O(N) where N is the number of active unique users/API keys.

Optimizations: Use a "fail-open" mechanism (if the rate limiter is down, allow traffic) to protect availability. Use Bloom Filters to quickly identify new/unknown keys before allocating counter memory.

Hints: How do you handle clock skew between edge nodes? What happens if the network between a specific region and the global store is partitioned?

Q2: Distributed Deduplication and Reference Tracking for Azure Container Registry (ACR)

Difficulty: Hard

Tags: Storage, Hashing, Cloud Infrastructure, Metadata Management

Problem Statement: ACR stores millions of container images. Many layers (blobs) are identical across different images and users. Design a system that ensures unique layers are stored only once (Content Addressable Storage) and supports safe, atomic deletion of layers only when they are no longer referenced by any image manifest.

Expected Approach: Use SHA-256 hashing to identify blobs. Store blobs in a central object store. Maintain a relational database table that maps ImageID to BlobID. Use a simple "Reference Count" column in the Blob table that increments on upload and decrements on delete.

Optimized Approach: Implement a Two-Phase Garbage Collection (GC) system to avoid the "Delete-While-Uploading" race condition. Phase 1: Mark-and-Sweep. Identify all blobs referenced by active manifests and mark them. Phase 2: Sweep unreferenced blobs older than 24 hours. For metadata, use a Distributed Key-Value store with "Compare-and-Swap" (CAS) operations to manage reference logs. Use Bloom Filters at the ingestion tier to bypass storage checks for existing blobs.

Time Complexity: O(1) for existence checks; O(M + B) for GC cycles (M=manifests, B=blobs).

Space Complexity: O(U) where U is the number of unique blobs.

Optimizations: Implement "Virtual Deletion" (soft deletes) to allow for immediate recovery of accidentally deleted layers and to reduce I/O spikes during GC.

Hints: What happens if a manifest is uploaded referencing a blob that is currently being deleted by the GC sweep? How do you scale the metadata store if the number of references exceeds billions?

Q3: Global Quota Management for Azure Multi-Region Services

Difficulty: Hard

Tags: Distributed Systems, Quota Leasing, Consistency vs Availability

Problem Statement: Design a system to enforce a global rate limit of N requests per second for a specific API across 10 different geographical regions. The solution must ensure that the global limit is strictly enforced (no more than 1% burst overage) while minimizing the cross-region latency penalty for every request.

Expected Approach: Use a centralized Redis cluster to track global counts. Every request from any region performs an atomic increment (`INCR`) and check.

Optimized Approach: Implement a "Quota Leasing" mechanism. Each regional proxy requests a "lease" (a batch of tokens) from the global coordinator. The lease size is dynamically adjusted based on the region's current request velocity (e.g., a region handling 50% of traffic gets 50% of tokens). Local regions consume their lease offline. Only when a local lease is near exhaustion does the region fetch a new batch from the global coordinator.

Time Complexity: O(1) for local token consumption; O(1) for periodic background lease renewal.

Space Complexity: O(T) where T is the number of active tenants/API keys stored in memory.

Optimizations: Use a "Predictive Demand" algorithm (Exponential Moving Average) to request larger leases during traffic spikes and smaller leases during troughs to reduce network overhead.

Hints: How do you handle a total network partition between a region and the coordinator? Should you fail-open or fail-closed?

Q4: High-Performance Dependency Re-evaluation in Excel Online

Difficulty: Hard

Tags: Graph Theory, Incremental Computation, Topological Sort

Problem Statement: In a spreadsheet with 10^6 cells, many cells contain formulas referencing other cells (forming a Directed Acyclic Graph). When a user updates a single cell, you must re-calculate all affected cells in the correct order. You must handle complex chains and ensure that no cell is recalculated more than once.

Expected Approach: Build an adjacency list representing the dependency graph. When a cell X changes, perform a Depth First Search (DFS) or Breadth First Search (BFS) to find all reachable nodes and recalculate them.

Optimized Approach: Use a "Level-Based Incremental Evaluation." Assign each node a "rank" representing its maximum distance from any leaf node. When a cell changes, identify all affected nodes using a forward sweep. Sort the affected nodes by their rank and evaluate them in increasing order. This ensures that when a cell is evaluated, all its parent dependencies have already been updated. Use a versioning system (Dirty Bits) to skip branches of the graph that haven't actually changed in value.

Time Complexity: O(V + E) for dependency discovery; O(K log K) for sorting K affected nodes.

Space Complexity: O(V + E) to store the dependency graph in memory.

Optimizations: Implement "Cycle Detection" during formula entry using Tarjan’s algorithm to prevent infinite loops before they are committed to the graph.

Hints: If a cell's value doesn't change after re-calculation, do you still need to update its children? How do you handle "Volatile Functions" like RAND()?

Q5: Multi-Tenant Global Quota Management (Distributed Rate Limiting)

Difficulty: Hard

Tags: Distributed Systems, Scalability, Redis, Concurrency

Problem Statement: Azure API Management must enforce a global rate limit of N requests per second for a subscription across 50+ geographic regions. A centralized database for every request introduces high latency (300ms+), while purely local counters allow users to exceed their global quota by a factor of 50. Design a system that ensures strictness within a 5% error margin while maintaining sub-10ms latency.

Expected Approach: Use a centralized Redis cluster to increment a global counter on every request. This ensures accuracy but fails latency requirements due to cross-region round-trips.

Optimized Approach: Implement a "Hierarchical Token Lease" system. Each regional edge node requests a "batch" of tokens (e.g., 500 tokens or a 1-second supply) from a global coordinator. The edge node handles requests locally using these tokens. When the local buffer hits a "low-water mark" (e.g., 10% remaining), it asynchronously requests a new lease.

Time Complexity: O(1) for request processing; O(1) for periodic background synchronization.

Space Complexity: O(S * R) where S is the number of active subscriptions and R is the number of regions.

Optimizations: Use "Traffic-Based Weighting" to allocate larger leases to regions with higher historical RPS to minimize synchronization frequency.

Hints: How can we handle a sudden burst in one region if the tokens are locked in another? Think about "Pre-emptive Rebalancing."

Q6: Efficient Delta Sync for Large OneDrive Binary Blobs

Difficulty: Hard

Tags: Rolling Hash, Algorithms, Data Compression, Merkle Trees

Problem Statement: OneDrive needs to sync large virtual machine disk images (e.g., 100GB .vhd files). If a user modifies a single sector, uploading the entire file is too expensive. Design a mechanism to identify and upload only the changed bytes (deltas) without requiring the client to store a full secondary copy of the file for comparison.

Expected Approach: Divide the file into fixed-size 4KB blocks and generate a MD5/SHA-256 hash for each. Compare hashes with the server's version. If a byte is inserted at the start, every subsequent block hash changes, causing a full re-upload.

Optimized Approach: Use Content-Defined Chunking (CDC) with a Rolling Hash (Rabin Fingerprint). Slide a small window (e.g., 48 bytes) across the file. Calculate the hash at every byte. A chunk boundary is created only when `hash % AverageChunkSize == MagicNumber`. Since boundaries are determined by content, an insertion only affects the chunk it occurs in, leaving other chunk hashes identical.

Time Complexity: O(N) to scan the file once.

Space Complexity: O(N/K) where K is the average chunk size to store signatures.

Optimizations: Use a Merkle Tree (Hash Tree) of the chunk signatures to allow the server and client to quickly identify which specific sub-sections of a massive file differ.

Hints: Think about how "rsync" works. How do we ensure the chunk sizes don't become too small or too large? (Min/Max chunk constraints).

Q7: Azure Spot Instance Interruptible Task Scheduler

Difficulty: Hard

Tags: Greedy, Priority Queue, Sliding Window

Problem Statement: You are designing a scheduler for Azure Spot Instances. You have a single long-running batch job that requires T total compute units to complete. You are given an array of prices P where $P[i]$ is the cost of compute for hour i, and an array M where $M[i]$ is the probability of the instance being reclaimed (interrupted) by Azure at hour i. You have a budget B. If an instance is reclaimed, all work done in that specific hour is lost, but work from previous hours is saved. Find the starting hour S and the duration D that minimizes the total cost to complete T units of work while staying within budget B and ensuring the cumulative probability of interruption does not exceed a threshold K.

Expected Approach: A brute-force approach would check every possible start time and calculate the expected cost and interruption probability for all possible durations until T units are accumulated. This results in $O(N^2)$ complexity, which is too slow for large time horizons.

Optimized Approach: Use a sliding window combined with a prefix sum array for costs and a logarithmic transformation for interruption probabilities. To calculate the probability of *not* being interrupted over a range $[i, j]$, use the formula $1 - \prod (1 - M[k])$. By taking the log of $(1 - M[k])$, the product becomes a sum, allowing for $O(1)$ range queries. Use a Two-Pointer/Sliding Window to find the shortest duration D starting at each hour i that meets the work requirement T and the risk threshold K.

Time Complexity: O(N)

Space Complexity: O(N)

Optimizations: Pre-calculate the prefix sums of costs and the prefix sums of $\log(1 - M[i])$ to allow for constant time validation of any given window $[i, j]$.

Hints: How can you convert a product of probabilities into a summation? Consider how the "lost work" constraint affects the total duration needed to reach T.

Q8: Microsoft Teams: Collaborative Range Locking Engine

Difficulty: Medium/Hard

Tags: Segment Tree, Interval Merging, Concurrency

Problem Statement: In a collaborative Excel-online session, users can "lock" a range of cells $[R_1, C_1]$ to $[R_2, C_2]$ to prevent edits. Given a stream of lock requests $(UserID, StartCell, EndCell, Timestamp)$, where cells are mapped to a 1D coordinate system $[L, R]$, implement a system that: 1. Grants a lock if it doesn't overlap with any existing locks. 2. Automatically releases a lock after X minutes. 3. Provides a function to find the "Most Contested" cell (the cell that had the highest number of rejected lock requests).

Expected Approach: Use a Hash Map to store active locks and check for overlaps by iterating through all active intervals. For the "Most Contested" cell, keep a frequency map of all cells within a rejected range. This approach is $O(N)$ per request and $O(CellRange)$ for contested tracking, which is inefficient for large grids.

Optimized Approach: Use a Segment Tree with Lazy Propagation to manage the 1D interval space. Each node in the tree stores whether the range is locked. For the "Most Contested" cell, use a separate Segment Tree or a Fenwick Tree to record "hits" on rejected ranges. To handle the X minute expiration, use a Priority Queue (Min-Heap) storing `(expiration_time, interval)` to proactively or lazily remove expired locks from the Segment Tree.

Time Complexity: O(log N) per lock request/release.

Space Complexity: O(N) where N is the number of active locks or the coordinate space.

Optimizations: Use coordinate compression if the cell range is sparse (e.g., 10^9 cells but only 10^5 lock requests) to keep the Segment Tree size manageable.

Hints: Think about how to represent a 2D grid as a 1D line. How does a Segment Tree help with range updates versus point queries?

Q9: Azure Blob Storage: Deduplication via Content-Defined Chunking

Difficulty: Hard

Tags: Rolling Hash, Data Deduplication, Rabin-Karp, Systems Design

Problem Statement: Azure Storage needs to minimize costs by avoiding storing duplicate data across millions of files. Traditional fixed-size chunking (e.g., every 4KB) fails if a single byte is inserted at the start of a file, as all subsequent boundaries shift. Design a mechanism to identify duplicate data blocks even when offsets shift, allowing for "delta" storage.

Expected Approach: Use a rolling hash (like Rabin-Karp) to scan file content. Define a "breakpoint" condition (e.g., the last n bits of the hash equal a specific value). When the condition is met, a chunk boundary is created. This ensures boundaries are based on content, not fixed offsets.

Optimized Approach: Implement a sliding window with a polynomial rolling hash. Use a Content-Defined Chunking (CDC) algorithm (e.g., Gear Hash or FastCDC) to reduce CPU overhead. Store chunk hashes in a distributed Key-Value store (like CosmosDB) to check for global existence before writing.

Time Complexity: O(N) where N is the total number of bytes in the file.

Space Complexity: O(M) where M is the number of unique chunks identified.

Optimizations: Use bitwise shifts instead of modular arithmetic for the rolling hash; implement a " minimum" and "maximum" chunk size to prevent extremely small/large segments.

Hints: How would you ensure that a small change in the middle of a 1GB file doesn't force a full re-upload? Think about boundaries that "stick" to the data pattern.

Q10: Excel Online: Dynamic Dependency Recalculation Engine

Difficulty: Medium-Hard

Tags: Directed Acyclic Graph (DAG), Topological Sort, Memoization

Problem Statement: In Excel Online, a cell formula can depend on other cells (e.g., A1 = B1 + C1). When a user updates a cell, you must update all dependent cells in the correct order. If A1 depends on B1, and B1 depends on C1, updating C1 must trigger B1 then A1. Additionally, you must detect and prevent circular dependencies (e.g., A1 -> B1 -> A1).

Expected Approach: Represent the spreadsheet as a Directed Graph where nodes are cells and edges are dependencies. Perform a Depth First Search (DFS) from the modified cell to identify all reachable nodes. Use a state-based approach (Unvisited, Visiting, Visited) to detect cycles.

Optimized Approach: Maintain an adjacency list of "dependents" for each cell. When a cell changes, perform a Topological Sort on the affected sub-graph using Kahn's Algorithm or DFS. Use "Dirty Bits" to mark cells needing update, and only recompute those whose direct inputs have changed to handle massive sheets efficiently.

Time Complexity: O(V + E) for cycle detection and sort; O(C) for recalculation where C is the number of affected cells.

Space Complexity: O(V + E) to store the dependency graph in memory.

Optimizations: Implement "Lazy Evaluation" for hidden sheets; use a thread pool to calculate independent branches of the DAG in parallel.

Hints: What happens if the user defines a massive chain of 10,000 cells? How do you avoid re-calculating the entire sheet if only one value changed?
